

DNS, TLS i mTLS w Kubernetes

Jak opowiedzieć o nazwach, certyfikatach i szyfrowaniu ruchu

DNS: gdzie jest usługa?

TLS: czy ufam endpointowi?



DNS: dwa światy

DNS wewnętrzny Kubernetes

Service → Pod
CoreDNS
namespace
svc.cluster.local

DNS publiczny

app.example.com
LoadBalancer / Ingress
użytkownik z internetu
rekord A/CNAME

CoreDNS nie jest publicznym DNS-em dla użytkowników internetu.



DNS w klastrze — demo

```
kubectl run dns-test -n qa-demo \
  --image=busybox:1.36 --rm -it --restart=Never -- sh

cat /etc/resolv.conf
nslookup nginx-secret-demo
nslookup nginx-secret-demo.qa-demo.svc.cluster.local
```

Krótką nazwą działa w tym samym namespace.

Pełną nazwą działa jawnie między namespace.



Klasyczny TLS: HTTPS kończy się na Ingressie



To najczęstszy model: użytkownik ma HTTPS do Ingressa, a ruch dalej w klastrze może iść jako HTTP.



mTLS: dodatkowa warstwa szyfrowania w klastrze



Przy mTLS HTTPS może nadal kończyć się na Ingressie, ale ruch wewnątrz klastra jest ponownie szyfrowany i uwierzytelniany przez mesh.

TLS z internetu i mTLS między usługami to często dwie różne warstwy.



A kiedy TLS nie kończy się na Ingressie?

Ingress może robić TLS passthrough
certyfikat może być terminowany dopiero w aplikacji
część mesh/gateway może pracować w trybie pass-through
model zależy od architektury, polityk bezpieczeństwa i narzędzi

**Dlatego przy pytaniu „gdzie kończy się HTTPS?” odpowiedź brzmi:
trzeba sprawdzić konkretną konfigurację.**



Jakie certyfikaty mogą istnieć?

cluster certs

API server, kubelet, etcd, komponenty klastra

kubeconfig

certyfikat klienta albo token użytkownika

Ingress TLS

certyfikat aplikacji dla domeny publicznej

mesh mTLS

krótkie certyfikaty workloadów wydawane przez mesh CA



Renew certów: najpierw ustalamy które

certyfikaty klastra: zależne od dystrybucji — kubeadm, k3s, Talos, cloud

Ingress TLS: zwykle cert-manager, ACME albo firmowe CA

mesh mTLS: zwykle automatyczna rotacja certyfikatów workloadów

kubeconfig użytkownika: nowy kubeconfig albo odnowienie client cert/tokena

```
sudo k3s certificate check --output table  
kubectl get certificate -A  
kubectl get secret <tls-secret> -n <ns> -o yaml
```



Troubleshooting: DNS, TLS, mTLS

DNS

```
nslookup <service>
kubectl get svc,endpoints -n <ns>
kubectl -n kube-system get pods |
grep coredns
```

TLS

```
openssl s_client -connect host:443 -
servername host
kubectl get secret <tls-secret> -n <ns>
kubectl describe certificate <name> -n <ns>
```

Przy mTLS dodatkowo sprawdzamy sidecary, polityki mesh i certyfikaty workloadów.



Do zapamiętania

DNS

czy nazwa rozwiązuje się na właściwą usługę?

TLS

gdzie kończy się HTTPS i kto ma certyfikat?

mTLS

czy ruch między usługami jest szyfrowany i uwierzytelniany?

Najważniejsze pytanie przy certach: o który certyfikat chodzi?

